

Smart Distress Signal Detector

Frank Tamburro

*Electrical and Computer Engineering
Lehigh University
Bethlehem, PA
fjt227@lehigh.edu*

Alex Hume

*Electrical and Computer Engineering
Lehigh University
Bethlehem, PA
alh522@lehigh.edu*

Abstract—This paper presents the design and implementation of a prototype Smart Distress Signal Detector system built using a Tiva microcontroller.

I. SYSTEM OBJECTIVE INTRODUCTION

A. Overall System

The Smart Distress Signal Detector is a system designed to passively monitor the local environment for distress events based on audio patterns and respond with visual, mechanical, and digital alerts. The objective of the fully scaled system is to aid individuals who may be unable to manually trigger an emergency call, such as elderly individuals, people with mobility impairments, or even home intrusion. A fully scaled deployment of the Smart Distress Signal Detector would include language recognition, GPS tracking, cross-system connectivity via the internet/cellular, and a mobile app for adjusting and managing the system.

B. Prototype

The prototype implements the core of the distress detection logic and alert mechanisms on a TM4C123 microcontroller. To define distress events, it detects repeated loud bursts of audio within a customizable time window if a person is detected within proximity of an IR sensor. The alert responses are prototyped via LEDs, a servo-driven flag, and event logging via UART. Ultimately, the prototype demonstrates the behavior of the complete system via passively detecting distress patterns without physical interaction from an individual in distress.

II. SYSTEM SPECIFICATIONS AND FUNCTIONALITY REQUIREMENTS

A. Overall System

The Smart Distress Signal Detector must passively detect audio patterns without requiring any physical interaction from the individual in distress. The system must notify emergency services upon detection of a distress event. Remote configuration and monitoring must be supported through a dedicated mobile application, allowing authorized users to adjust system sensitivity. The system must support multiple microphones and IR sensors to provide good coverage across larger spaces. Language recognition must be incorporated to identify specific distress sounds, and GPS tracking must be included to report the location of a detected event. Connection via cellular and internet networks is required to facilitate alert transmission.

From an ethical standpoint, the system must be designed to minimize false positives as an unnecessary emergency response poses risk to both individual and emergency personal. Since the system relies on audio monitoring, a fully deployed implementation must comply with applicable recording consent laws and notify individuals of its presence to preserve privacy.

B. Prototype

The prototype must implement three operating states: IDLE, MONITOR, and ALERT. System inputs must include an IR sensor for person detection, a microphone producing a digital output for audio sampling, a reset button for returning the system to IDLE from any state, and an ADC-based potentiometer for configuring the distress detection sensitivity threshold. Audio processing must only occur when a person is detected in proximity, ensuring the system does not evaluate background noise when an individual is not there. The sensitivity of distress detection must be adjustable across three levels, controlled by the potentiometer. Outputs must include a green LED to indicate the MONITOR state, a red LED to indicate the ALERT state, and a servo motor that raises a physical flag upon entering the ALERT state. The flag is lowered in any other state. Event logging must be transmitted to a serial terminal to provide visibility into the system activity including loud burst detections, distress events, and resets. The system must recover autonomously from software crashes without requiring manual intervention.

III. SYSTEM DESIGN

A. Overall System

The fully scaled Smart Distress Signal Detector is designed as an autonomous monitoring system capable of operating in residential and commercial environments. The system architecture consists of three primary layers: a sensing layer, a processing layer, and a communication layer. The sensing layer incorporates multiple microphones for audio capture, and many IR sensors for proximity detection. Also a language recognition module to identify specific distress noises. The processing layer analyzes incoming sensor data, applying pattern recognition to distinguish genuine distress events from surrounding noise. Upon detection of a distress event, the communication layer transmits an alert through cellular and internet connectivity to a mobile application, enabling remote

monitoring and emergency response coordination. The system is designed to operate continuously without requiring user interaction, making it suitable for individuals with mobility impairment or medical conditions that prevent manual emergency signaling.

B. Prototype

The prototype is designed around a Moore-style FSM (Finite State Machine) implemented on a TM4C123 microcontroller. The Moore style consists of three states: IDLE, MONITOR, and ALERT. The FSM transitions are driven by four possible inputs: no input, person detected, distress confirmed, and reset. Person detection is handled by an IR sensor on PF1 which prevents audio processing from occurring unless a person is confirmed to be in proximity. This reduces false positives from environmental noise. The microphone digital output on PE5 is sampled on every SysTick, and an adjustable window of 100 ticks is used to count the number of high samples in within that window. Distress detection sensitivity is adjustable with a potentiometer on PE3, which is read through ADC (Analog to Digital Conversion). The ADC value is mapped to one of the three burst thresholds: 2, 4, or 6, which correspond to low, medium, and high sensitivity respectively. This allows the system to be tuned for different environments according to the users preferences. When the burst count meets the configured threshold, the FSM transitions from MONITOR to ALERT. In the ALERT state, a red LED on PE1 activates and the servo motor on PE4 rotates 180°, representing a physical flag raise to provide a visible distress indicator. A green LED on PE2 remains active during the MONITOR state to indicate the system is actively tracking a person. Event logging is output to a serial terminal via UART providing real time visibility into system events. A watchdog timer is configured to reset the system if the the SysTick ISR does not feed it within the defined reload period.

IV. IMPLEMENTATION EXPLANATION

The implementation of the prototyped system began by defining constants and global variables required across the system. Pin hex values for the LEDs, PWM constants for the servo motor, and the SysTick reload constant were among the definitions. Namely, the **WINDOW_SIZE** constant, defined the fixed 100 ticks that made-up the evaluation window. Global variables were utilized across functions, especially for audio detection purposes. More specifically, the **high_ticks_target** variable designated that the microphone pin needed to be high for at least 30 of the 100 ticks in a window to qualify as a loud event. While, the **burst_target** variable designates the number of qualifying windows as designated by the potentiometer, or loud events, that must occur before the distress is triggered. Four additional global variables were also essential in designing the audio detection: **high_tick_count** and **window_counter** were written every tick in **mic_isr()** to track the current window, while **sound_count** counted qualified windows and **loud_event_num** did similarly for UART purposes. However, the distinction is important as

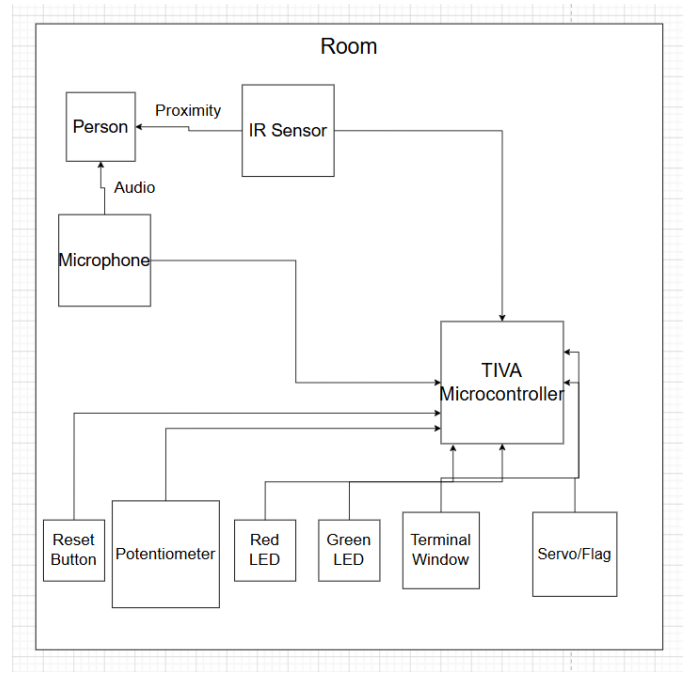


Fig. 1. Diagram of Smart Distress Signal Detector

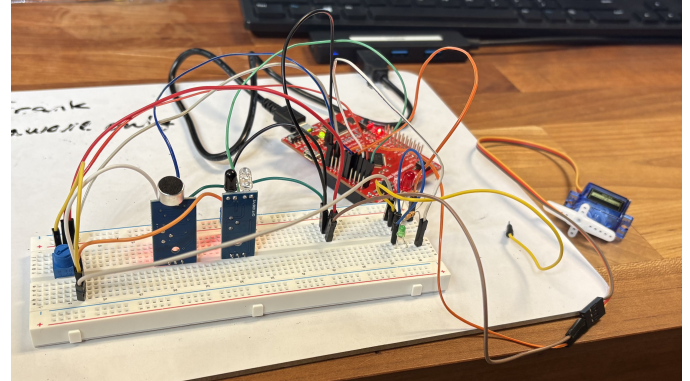


Fig. 2. Image of Smart Distress Signal Detector

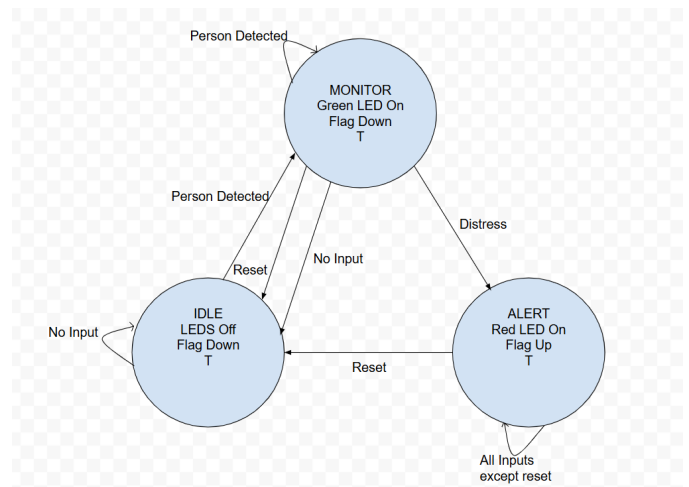


Fig. 3. FSM

sound_count is compared against **burst_target** to trigger a distress event and resets immediately after, while **loud_event_num** is only used for UART and resets separately. The **loud_event_flag** and **tick_flag** variables were used to help ensure timely audio detection and delay, respectively. The **g_bWatchdogFeed** variable was set by **mic_isr()** and read by **WatchdogIntHandler()**. The **button_prev** variable was used to handle timing interactions with the reset button.

Prior to the main function, the Moore-style FSM was defined using a struct that contained: the output bitmask **out**, the delay period **T**, and the next state array of 4 elements, **next[4]**. The **fsm[3]** consisted of 3 states (IDLE, MONITOR, ALERT). The **out** field bit masks depended solely on the current state of the system **cstate** and would drive the green LED and red LED for the MONITOR and ALERT states, respectively. The next state array was indexed by 4 possible inputs: no input, person detected, distress confirmed, and reset button.

The **main** function began by setting up the system. This involved setting the system clock to 50MHz and initializing the input and output peripherals using custom functions. The **portF_input_setup()** function was called consecutively to configure PF0 for the reset button SW2 and PF1 for the IR sensor, which are both active-low inputs. The **portE_input_setup()** function was called in a similar manner to set PE5 as an input for the microphone. On the other hand, the **portE_output_function()** function was used to initialize PE1 (Red LED) and PE2 (Green LED) as outputs by setting their GPIO port direction. The ADC was also initialized via the **adc_setup()** function and configured to PE3. Additionally, the servo motor was setup and configured to PE4 via the **servo_pwm_setup()** function. The UART was initialized by **uart_setup()** and allowed functionality with printing into the device terminal. The SysTick timer was configured and set by its reload value **T**. Furthermore, the **mic_isr()** function was set as the SysTick ISR and global interrupts were enabled. The Watchdog timer was initialized last via **watchdog_setup()** to ensure all peripherals were configured beforehand. Finally, before the infinite loop, **cstate** and **sound_count** were set to 0 for a fresh start of the system.

Within the infinite while loop, the FSM was executed each iteration. First, **update_output()** was called based on the current states **out** bit mask, which determined the states of the green LED, red LED, and servo motor, via the **servo_set_angle()** function. The delay was then tracked by **tick_flag** which was set in **mic_isr()** and was dependent on **T**, the SysTick reload value. Next, the **adc_read_threshold()** function triggered an ADC sample to set **burst_target** to either SHORT, MEDIUM, or LONG based on the **adc_raw** value. **uart_print_window()** was only called when the level changed, which was determined by comparing **burst_target** to **prev_target**. Fourth, if the **loud_event_flag** was set by **mic_isr()** then **uart_print_loud_event()** was called to print the **loud_event_num**. All inputs were then evaluated in **get_input()** and indexed in the FSM table where the reset button was held with the highest priority. There were also

conditionals checking the input index in order to call the corresponding UART print function: either **uart_print_distress()** or **uart_print_reset()**. Finally, **cstate** was updated by indexing the FSM table.

The **mic_isr()** function acted as the SysTick ISR and completed three tasks each iteration, or tick. Firstly, it set **tick_flag** to true and **g_bWatchdogFeed** to 1 in order to signal the system was functioning. Secondly, it checked the state of IR sensor to ensure a person was still in proximity, otherwise counts were set to 0 to avoid false audio detections. Finally, the microphone was sampled from PE5 and incremented **window_counter**, and **high_tick_count** if high. When **window_counter** reached the set **WINDOW_SIZE**, it evaluated if **high_tick_count** was greater than or equal to **high_ticks_target** to qualify a loud event. If so, **sound_count** and **loud_event_num** were both incremented, while **high_tick_count** and **window_counter** were set to 0 regardless.

The **WatchdogIntHandler()** ISR fired whenever the watchdog timer expired. If the watchdog flag was 1, the handler called **WatchdogIntClear()** to reset the clock and set the flag to 0 such that **mic_isr()** fed it again. If the flag **g_bWatchdogFeed** was 0, the handler performed a soft reset on the system by resetting all outputs, the current state, and clearing all variables. The watchdog timer system was enabled and configured in the **watchdog_setup()** function.

V. TESTING PROCEDURE

The testing procedure of the prototype involved testing multiple situations that simulated the behavior of the actual Smart Distress Signal Detector system. Each test was used to simulate and confirm the intended behavior of the system. Testing was completed incrementally throughout implementation to verify functionality of each subsystem.

LEDs: verify the green LED only activates during MONITOR state and person is detected by IR sensor, and verify red LED activates when ALERT mode (green LED off)

IR Sensor: verify IR sensor triggers transition from IDLE to MONITOR as indicated by green LED

Potentiometer and ADC: verify rotation of full range of potentiometer results in accurate updated print statements in UART terminal: "WINDOW: SHORT," "WINDOW: MEDIUM," and "WINDOW: LONG"

Audio Detection: verify when IR sensor is active that loud events are processes accurately and printed in UART terminal (ex. "LOUD EVENT #: 1") with account for window sensitivity (burst targets of 2, 4, 6 for SHORT, MEDIUM, and LONG, respectively)

Servo Motor: verify rotation as the result of distress event and reset upon IDLE state

Reset Button: verify reset button sends system to IDLE and resets all outputs, regardless of current state

Full System: verify all subsystems interact in accordance with prototype specifications and full IDLE, MONITOR, and ALERT state transition with functional distress signal

VI. RESULTS

The prototyped Smart Distress Signal Detector was tested by systematically transitioning through the system's states and verifying the expected outputs with accurate readings at each state. All visual, mechanical, and logging outputs performed as expected with respect to the system's intended functionality and testing procedure.

The Moore-style FSM that defined the transition logic of the system correctly transitioned through the IDLE, MONITOR, and ALERT states based solely on the input and current state. All outputs, the two LEDs and servo motor, accurately behaved within their respective states, with the green LED on during MONITOR, the red LED and servo motor active during ALERT, and no outputs during IDLE. Additionally, the IR sensor was able to reliably detect the presence of a person, allowing the system to transition between the MONITOR and IDLE states depending on the proximity detection. The reset push button, SW2, also consistently returned the system to the IDLE state regardless of the current state.

The audio detection via the microphone peripheral was able to be accurately read into the system through a count of high tick readings defined within an adjustable window size. The ADC-based potentiometer correctly adjusted the sensitivity of the window by adjusting the trigger thresholds for a loud event across three settings: SHORT (2 seconds), MEDIUM (4 seconds), and LONG (6 seconds). Furthermore, the UART output print statements in the terminal accurately demonstrated the adjustable features and reading status of the system, which provided a clean trace of the system. The watchdog implementation was also successful in setting an external timer to interrupt any system malfunctions and demonstrated a soft reset of the system, if so. Overall, the system ran continuously without any expected resets or incorrect outputs and demonstrated consistent stability.

VII. CONCLUSION

A. Overall System

A fully scaled implementation of the Smart Distress Signal Detector could have meaningful impact on improving safety outcomes by providing passive, hands-free emergency detection. In a complete system, language recognition, cellular connection for external alerts, and GPS location reporting would improve the effectiveness of the system. Additionally, ethical concerns, such as continuous audio monitoring, must be transparent and protect individual's privacy with the aim for the system to be lifesaving device, rather than a surveillance asset. Ultimately, the prototype confirms the distress detection capabilities and implies a complete system could be deployed at scale.

B. Prototype

The prototype successfully demonstrates a functional distress detection system on a Tiva TM4C123 microcontroller. The implementation correctly used a Moore-style FSM to transition between states while applying other design criteria such as the ADC, UART, watchdog timer, timely SysTick

interrupts, multiple LED outputs, IR sensor, servo motor, reset button, and microphone peripheral to build a complete prototyped system. The prototype reliably detected sustained loud audio events when a person is in proximity, triggered visual and physical alerts, logged all system events in UART, and manual resets, that of which validates the core goal of passive distress detection. Key design considerations needed to be made in the implementation of the system, such as the window-based audio detection and the use of the ADC-based potentiometer to configure the sensitivity of the windows.